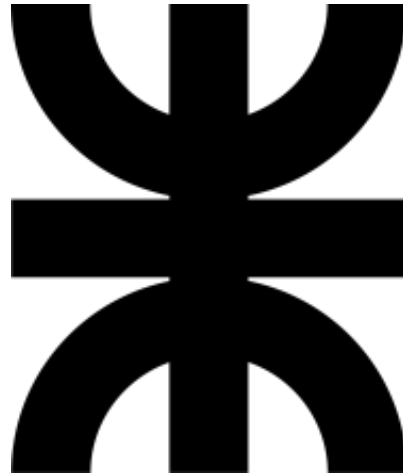


Universidad Tecnológica Nacional

Facultad Regional Córdoba

Ingeniería en Sistemas de Información
Administración de Sistemas de Información



TP 6 - Test Driven Development (TDD)

Año: 2025

Docentes:

- Ing. Laura Covaro
- Ing. Cecilia Massano
- Ing. Constanza Garnero

Curso: 4K3

Grupo Nro: 14

Integrantes:

- Riera, Martin Fernando, 91746
- González Bernahola, Alessandro Tomas, 92950
- Sangenis Libra, Valentino, 90153
- Magris, Santino Alejandro, 91999
- Haro Monforte, Tomás, 83204
- Caliva, Ariel Enrique, 69777
- Patriarca, Ignacio, 91025
- Simes, Juan Mateo, 96074
- Dilewski, Ignacio Nicolás, 97662

Implementación — Inscripción a Actividades (EcoHarmony Park)

1. Resumen

Se implementó la funcionalidad “**Inscribirme a actividad**” siguiendo el enfoque TDD (Red → Green → Refactor).

Stack principal: **Python (FastAPI / uvicorn)** en el backend, **SQLite** (DatabaseSingleton) para persistencia y tests con **unittest**. Además, se desarrolló una interfaz frontend con **Next.js** para la interacción del usuario.

La solución cubre selección de actividad y horario, registro de participantes, decremento de cupos, aceptación de términos por actividad y validaciones (nombre sin números; DNI solo dígitos). La API expone los endpoints necesarios para que el front consuma la información de actividades y horarios y realice inscripciones.

2. Alcance funcional (User Story principal)

Como visitante

Quiero inscribirme a una actividad

Para reservar mi lugar.

Criterios implementados:

- Seleccionar una actividad del catálogo (Tirolesa, Safari, Palestra, Jardinería) — solo si hay cupos en el horario.
- Seleccionar un horario disponible (cupos por horario).
- Indicar cantidad de participantes y completar, por cada uno: nombre, DNI, edad y talla si la actividad lo requiere.
- Requerir aceptación de términos y condiciones específicos de la actividad.
- Validaciones: nombre (solo letras y espacios), DNI (solo dígitos).
- Persistencia de participante e inscripción; decremento de cupos en la tabla **horario**.
- Respuesta clara al front con éxito/fallo y motivos en caso de error.

3. Arquitectura (resumen)

- Patrón: **MVC** (carpetas separadas):
 - Persistencia/ → repositorios y DatabaseSingleton.
 - Servicio/ → lógica de negocio (InscripcionService).
 - Modelo/ → clases de dominio.
 - backend/app.py → controladores (FastAPI).
 - front/ → Next.js.
- Base de datos: **SQLite** manejada por DatabaseSingleton.
- Tests: back/tests/ con unittest.
- Integración: REST JSON; CORS configurado en FastAPI para permitir el front local.

4. Tests (unittest) — cómo correr y qué cubren

Ubicación de la suite: back/tests/testsInscripcion.py (o back/tests/ si tenés varios).

Qué cubren (resumen):

- Inscripción exitosa (cupos decrementan y registro creado).
- Falla cuando no hay cupos.
- Falla si no se aceptan términos.
- Inscripción válida sin talle cuando la actividad no lo requiere.
- Validación de nombre inválido (contiene números).
- Validación de DNI inválido (contiene letras).

5. Pasos rápidos para el revisor

Activar virtualenv (opcional pero recomendado) e instalar dependencias:

```
cd back  
# Windows PowerShell ejemplo:  
.venv\Scripts\Activate  
pip install -r requirements.txt
```

Levantar el backend:

```
# si estás en back/  
cd src  
uvicorn app:app --reload --port 8000  
Verificá que /api/actividades responde en el navegador o con curl:  
http://localhost:8000/api/actividades
```

Ejecutar tests (desde back/):

```
python -m unittest discover -s tests -v
```

Levantar el front:

```
# Si estas en front/  
yarn start
```